

Unit 14: Storage Classes

CONTENTS

Objectives

Introduction

14.1 Storage Classes and their Usage

14.2 Automatic Variable

14.3 External Variable

14.4 External Declaration

14.5 Static Variable

14.6 Register Variable

14.7 Const Qualifier

Summary

Keywords

Self Assessment

Answers for Self Assessment

Review Questions

Further Readings

Objectives

After studying this unit, you will be able to:

- storage classes
- Scope of a variable
- Auto, Static, Extern and Register

Introduction

Storage Classes are used to characterise a variable's or function's characteristics. These characteristics include scope, visibility, and life-time, which allow us to track the presence of a variable through the course of a program's execution. The following items are described by a storage class in C:

The variable scope.

The location where the variable will be stored.

The initialized value of a variable.

A lifetime of a variable.

Who can access a variable?

14.1 Storage Classes and their Usage

There are two different ways to characterize variables:

1. by data types

2. by storage class

Data types refers to the type of information while storage class refers to the life-time of a variable and its scope within the program.

A variable in C can have any one of the four storage classes.

1. Automatic variable
2. External variable
3. Static variable
4. Register variable

14.2 Automatic Variable

An automatic variable's scope is limited to the function in which it is declared. When the function is called, it is formed, and when the function is exited, it is automatically deleted. As a result, the name Automatic was chosen.

Local variables are variables defined with the auto storage class. The term "auto" refers to the automatic storage class. If a variable is not explicitly declared, it is in the auto storage class by default.

The scope of an auto variable is restricted to a single block. The access is destroyed once the control leaves the block. This means that the auto variable can only be accessed from the block in which it is declared.

A keyword auto is used to define an auto storage class. By default, an auto variable contains a garbage value.

By default, a variable declared inside a function with storage class specification is an automatic variable. Automatic variable values cannot be changed accidentally by what happens in some other functions in the program.



Example:

```
main()
{
    int m = 1000;
    function 2();
    printf ("%d \n", m);
}

function 1()
{
    int m = 10;
    printf ("%d \n", m);
}

function 2()
{
    int m = 100;
    function 1();
    printf ("%d \n", m);
}

output: 10
100
```

14.3 External Variable

An external variable is also known as a global variable. It is not confined to a single function. Its scope extends from the point of definition through the remainder of the program.

External variables can be accessed from any function that falls within their scope. They are declared outside a function. If a local variable and a global variable have the same name, local variable will have precedence over global in the function where it is declared.



Example:

```
: int count; main
{
count = 10;
-----
-----
-----
}
```

Notes

```
function ( )
{
int count = 0;
-----
-----
count ++;

}
```

When the function references the variable count, it will be referencing only its local variable, not the global one. The value of count in main() will not be affected.



Example:

```
/* illustration of working of global variable int x;
illustration of working of global variable
int x;
main( )
{
x = 10;
printf ("x = %d \n", x);
printf ("x = %d \n", fun1( ));
printf ("x = %d \n", fun2( ));
```

```

printf ("x = % d \n", func3());
}
fun1()
{
x = x + 10;
return x;
}
fun2()
{
int x = 1;
return x;
}
fun3()
{
x = x+10;
return (x);
}
Output: x = 10
x = 20
x = 1
x = 30

```

14.4 External Declaration

In the program segment discussed just previously, the main cannot access the variable y as it has been declared after the main function. This problem can be solved by declaring the variable with the storage class extern.



Example:

```

main()
{
extern int y; /* external declaration */
-----
}
fun1()
{
extern int y; /* external declaration */
-----
}
int y; /*definition */

```

The external declaration of y inside the functions informs the compiler that y is an integer type defined somewhere else in the program.

14.5 Static Variable

Static variables are defined within a function in the same manner as automatic variables, except that the variable declaration must begin with the static storage class designation.



Example: static int x; or static float y;

A static variable is initialized only once, when the program is compiled. It is never initialized again.

A static variable may be either an internal type or an external type, depending on the place of declaration. Internal static variables are those which are declared inside a function. The scope of internal static variables extends upto the end of the function in which they are defined. Therefore, internal static variables are similar to auto variables, except that they remain in existence (alive) throughout the remaining program. Therefore, internal static variables can be used to retain values between function calls.



Example:

```
/* Illustration of static variable */
main()
{
    inti;
    for(i=1; i<=3; i++) stat();
}
stat()
{
    static int x = 0;
    x = x+1;
    printf("x = %d;\t", x);
}
```

Output: x = 1; x = 2; x = 3

An external static variable is declared outside of all functions and is available to all the functions in that program. The difference between a static external variable and a simple external variable is that the static external variable is available only within the file where it is defined while the simple external variable can be accessed by other files also.

14.6 Register Variable

We can tell the compiler that a variable should be kept in one of the machine's registers, instead of keeping in the memory (where normal variables are stored) since a register access is much faster than a memory access and keeping the frequently accessed variables in the register will lead to faster execution of programs.

For example, Loop control variables. This is done as given below:

```
register int count;
```

Since only a few variables can be placed in the register, it is important to carefully select the variables for this purpose. However, C will automatically convert register variables into non-register variables once the limit is reached.



Case Study:

Fundamentals of Computer and C Programming

When a function is written before main it can be called in the body of main. If it is written after main then in the declaration of main you have to write the prototype of the function. The prototype can also be written as a global declaration.

Program:

Case 1:

```
#include <stdio.h>

main ( )
{
    inti;
    void (int *k) // D
    i = 0;
    printf (" The value of i before call %d \n", i);
    f1 (&i); // A
    printf (" The value of i after call %d \n", i);
}

void (int *k) // B
{
    *k = *k + 10; // C
}
```

Case 2:

```
#include <stdio.h>

void (int *k) // B
{
    *k = *k + 10; // C
}

main ( )
{
    inti;
    i = 0;
    printf (" The value of i before call %d \n", i);
    f1 (&i); // A
    printf (" The value of i after call %d \n", i);
}
```

Case 3:

```
#include <stdio.h>

void f1(int *k) // B
{
    *k = *k + 10; // C
}

main ( )
{
    inti;
```

```
i = 0;

printf ("The value of i before call %d \n", i);
f1 (&i); // A
printf ("The value of i after call %d \n", i);
}
```

Explanation

In Case 1, the function is written after main, so you have to write the prototype definition in main as given in statement D.

In Case 2, the function is written above the function main, so during the compilation of main the reference of function f1 is resolved. So it is not necessary to write the prototype definition in main.

In Case 3, the prototype is written as a global declaration. So, during the compilation of main, all the function information is known.

Questions

1. Write a function which receives a float and an int from main(), finds the product of these two and returns the product which is printed through main().
2. Write a function that receives 5 integers and returns the sum, average and standard deviation of these numbers. Call this function from main() and print the results in main().

Write a function that receives marks received by a student in 3 subjects and returns the average and percentage of these marks. Call this function from main() and print



Lab Exercise:

Program to demonstrate different storage classes

```
#include <stdio.h>
```

```
// declaring the variable which is to be made extern
```

```
// an initial value can also be initialized to x
```

```
int x;
```

```
void autoStorageClass()
```

```
{
```

```
    printf("\nDemonstrating auto class\n\n");
```

```
    // declaring an auto variable (simply
```

```
    // writing "int a=32;" works as well)
```

```
    auto int a = 32;
```

```
    // printing the auto variable 'a'
```

```
    printf("Value of the variable 'a'"
```

```
        " declared as auto: %d\n",
```

```
        a);
```

```
    printf("-----");
```

```
}

void registerStorageClass()
{

    printf("\nDemonstrating register class\n\n");

    // declaring a register variable
    register char b = 'G';

    // printing the register variable 'b'
    printf("Value of the variable 'b'"
           " declared as register: %d\n",
           b);

    printf("-----");
}

void externStorageClass()
{

    printf("\nDemonstrating extern class\n\n");

    // telling the compiler that the variable
    // z is an extern variable and has been
    // defined elsewhere (above the main
    // function)
    extern int x;

    // printing the extern variables 'x'
    printf("Value of the variable 'x'"
           " declared as extern: %d\n",
           x);

    // value of extern variable x modified
    x = 2;

    // printing the modified values of
    // extern variables 'x'
    printf("Modified value of the variable 'x'"
           " declared as extern: %d\n",
```



```

        x);

    printf("-----");
}

void staticStorageClass()
{
    inti = 0;

    printf("\nDemonstrating static class\n\n");

    // using a static variable 'y'
    printf("Declaring 'y' as static inside the loop.\n"
        "But this declaration will occur only"
        " once as 'y' is static.\n"
        "If not, then every time the value of 'y' "
        "will be the declared value 5"
        " as in the case of variable 'p'\n");

    printf("\nLoop started:\n");

    for (i = 1; i < 5; i++) {

        // Declaring the static variable 'y'
        static int y = 5;

        // Declare a non-static variable 'p'
        int p = 10;

        // Incrementing the value of y and p by 1
        y++;
        p++;

        // printing value of y at each iteration
        printf("\nThe value of 'y', "
            "declared as static, in %d "
            "iteration is %d\n",
            i, y);

        // printing value of p at each iteration
        printf("The value of non-static variable 'p', "

```

```

        "in %d iteration is %d\n",
        i, p);
    }

    printf("\nLoop ended:\n");

    printf("-----");
}

int main()
{

    printf("A program to demonstrate"
           " Storage Classes in C\n\n");

    // To demonstrate auto Storage Class
    autoStorageClass();

    // To demonstrate register Storage Class
    registerStorageClass();

    // To demonstrate extern Storage Class
    externStorageClass();

    // To demonstrate static Storage Class
    staticStorageClass();

    // exiting
    printf("\n\nStorage Classes demonstrated");

    return 0;
}

```

14.7 Const Qualifier

To declare a variable constant, we use the const qualifier. That is, after the variable has been initialised, we cannot modify its value. Const offers a lot of advantages. If you have a constant value for PI, for example, you don't want any element of the programme to change that value. As a result, you should declare it as a const.

The compiler may store objects defined with const-qualified types in read-only memory, and if the address of a const object is never used in a programme, it may not be stored at all.

Summary

- Auto, extern, register, static are the four different storage classes in a C program.
- In this unit, we learnt about “storage classes”. A keyword auto is used to define an auto storage class.
- Extern storage class is used when we have global functions or variables which are shared between two or more files
- The static variables are used within function/ file as local static variables. They can also be used as a global variable
- Register is used to store the variable in CPU registers rather memory location for quick access.
- Storage class represents the scope and lifespan of a variable.
- It also tells who can access a variable and from where?

Keywords

Auto

Register

Extern

Static

Self Assessment

1. What is the initial value of register storage class specifier?
 - A. 0
 - B. Null
 - C. Garbage
 - D. Infinite
2. What is the scope of extern class specifier?
 - A. Within block
 - B. Within Program
 - C. Global Multiple files
 - D. None of the above
3. What is the scope of static class specifier?
 - A. Within block
 - B. Within Program
 - C. Global Multiple files
 - D. None of the above
4. Which is not a storage class?
 - A. Auto
 - B. Struct
 - C. Typedef
 - D. Static

5. What is the output of the following program?

```
#include<stdio.h>

int main()
{
    static int a = 3;
    printf("%d", a --);
    return 0;
}
```

- A. 0
- B. 1
- C. 2
- D. 3

6. What will be the output of the following program?

```
#include <stdio.h>

static int y = 1;

int main()
{
    static int z;
    printf("%d %d", y, z);
    return 0;
}
```

- A. Garbage value
- B. 0 0
- C. 1 0
- D. 1 1

7. In case of a conflict between the names of a local and global variable what happens?

- A. The global variable is given a priority.
- B. The local variable is given a priority.
- C. Which one will get a priority depends upon which one is defined first.
- D. The compiler reports an error.

8. Where will the space be allocated for an automatic storage class variable?

- A. In CPU register
- B. In memory as well as in CPU register
- C. In memory
- D. On disk.

9. What is the output of the program?

```
static int k;
```

```
int main()
{
    printf("%d", k);

    return 50;
}
```

- A. -1
- B. 50
- C. 0
- D. Compiler error

10. The statement below is a _____?

```
extern int a;
```

- A. Declaration
- B. Definition
- C. Initialization
- D. None of the above

11. An external variable is one-

- A. Which is globally accessible by all functions
- B. Which is declared outside the body of any function
- C. Which resides in the memory till the end of a program
- D. All of the above

12. Functions in C are always _____

- A. Internal
- B. External
- C. External and Internal are not valid terms for functions
- D. Both Internal and External

13. Global variables are _____

- A. External
- B. Internal
- C. Both internal and external
- D. None of above

14. Which storage class is used for faster execution?

- A. Register
- B. Auto
- C. Extern
- D. Static

15. Which of the following is default storage class?

- A. Register
- B. Auto
- C. Extern
- D. Static

Answers for Self Assessment

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. C | 2. C | 3. A | 4. B | 5. D |
| 6. C | 7. B | 8. C | 9. B | 10. A |
| 11. D | 12. B | 13. A | 14. A | 15. B |

Review Questions

1. Write a program to demonstrate static storage classes
2. What is significance of storage classes.
3. Write a program to demonstrate auto storage classes
4. Write a program to demonstrate extern storage classes
5. Write a program to demonstrate register storage classes



Further Readings

Books Ashok N. Kamthane, "Programming with ANCI & Turbo C", Pearson Education, Year of Publication, 2008

B.W. Kernighan and D.M. Ritchie, "The Programming Language", Prentice Hall of India, New Delhi

Byron Gottfried, "Programming With C", Tata McGraw Hill Publishing Company Limited, New Delhi

Greg W Scragg, Genesco Suny, Problem Solving with Computers, Jones and Bartlett, 1997.



Web links

www.en.wikipedia.org

www.web-source.net

www.webopedia.com

LOVELY PROFESSIONAL UNIVERSITY

Jalandhar-Delhi G.T. Road (NH-1)

Phagwara, Punjab (India)-144411

For Enquiry: +91-1824-521360

Fax.: +91-1824-506111

Email: odl@lpu.co.in